Pay-iLift

Project Planning & Solution Design — Deliverables 2 and 3

Purpose. Define a practical implementation backlog and a design baseline for the Pay-iLift proof of concept (POC), using the supplied software-engineering POC requirements as the baseline. Pay-iLift is intended to help commuters and drivers coordinate lifts along compatible routes and work schedules.

Scope rule. The POC prioritises secure registration and sign-in, commuter profile/onboarding, trip records, feedback, authorised review, notifications, and persistent MySQL data through an ASP.NET Core API. Live GPS, automated dispatch, fare calculation/payments, production transport integrations, and native mobile apps are outside the baseline POC unless the reviewer approves a scope change.

2. PLAN — Implementation Backlog

Deliverable increments are ordered to reduce dependency risk: confirm requirements first, then establish the API and database, add user-facing workflows, and finish with verification and deployment.

Increment	Deliverables	Dependencies / exit criteria
2.1 Requirements & acceptance criteria	Confirm user roles, user stories, data fields, validation rules, feedback lifecycle, and reviewer expectations. Record any proposed lift-matching additions separately.	No technical dependency. Exit: reviewer agrees the POC scope and acceptance criteria.
2.2 Repository & application skeleton	Create Git repository, React + TypeScript frontend, ASP.NET Core Web API, configuration, environment setup, and basic health/API documentation.	Depends on agreed stack and repository access. Exit: frontend and API run locally; secrets are not committed.
2.3 Data model & persistence	Define entities and relationships; configure EF Core and MySQL; add migrations, indexes, and synthetic seed data.	Depends on approved data model and database connection. Exit: data can be created, retrieved, and survives API restart.
2.4 Authentication & profile onboarding	Implement registration, sign-in/sign-out, protected routes/API endpoints, role checks, commuter profile, and onboarding status.	Depends on API skeleton and user schema. Exit: unauthenticated requests are blocked and users can only access their own profile.
2.5 Trips & commuter dashboard	Show onboarding status, trip list/details, schedule, route, operator/vehicle information, status, previous feedback, concerns, and notifications.	Depends on persistence, profile, and trip model/API. Exit: dashboard shows data returned by the API.
2.6 Feedback & review workflow	Allow feedback for completed trips; validate rating/category/comment; provide feedback history and authorised reviewer response, status updates, and resolution.	Depends on trip records, identity, ownership checks, and feedback schema. Exit: invalid/ineligible feedback is rejected; reviewer-only actions are protected.

Increment	Deliverables	Dependencies / exit criteria
2.7 Tests, deployment & handover	Add unit/API and frontend tests, security checks, README, setup instructions, architecture/design notes, deployment configuration, demo and technical walkthrough evidence.	Depends on end-to-end workflows being implemented. Exit: agreed tests pass and reviewer can run/review the POC.

Key dependency sequence

```
Scope & acceptance criteria
        ↓
Git repository + React/TypeScript + ASP.NET Core API
        ↓
Data model + EF Core + MySQL migrations
        ↓
Authentication + commuter profile
        ↓
Trips/dashboard → feedback/reviewer workflow → notifications
        ↓
Testing + README + deployment + demonstration
```

Suggested prioritisation: deliver a thin, testable vertical slice early (register → sign in → save profile → view API-backed dashboard), then expand to trips and feedback. Treat route-based driver/rider matching as a proposed extension, not a blocker for the baseline POC.

3. DESIGN — Architecture, Data Model & API

3.1 Architecture sketch

The browser communicates with the ASP.NET Core API over HTTPS using JSON. The API owns authentication, authorisation, validation, business rules, and persistence. The React application must never connect directly to MySQL.

React + TypeScript web client
Commuter pages • Reviewer/admin pages
Forms • Client-side validation • API client

↓ HTTPS / JSON ↓

ASP.NET Core Web API
Authentication • Role/resource authorisation
Controllers • Application services • Server validation

↓ Entity Framework Core ↓

MySQL database
Users • Profiles • Routes • Operators • Trips
Feedback • Responses • Notifications

3.2 Main data model

Entity	Purpose / important fields	Relationships
User	Identity-managed account: id, email, password hash managed by Identity, role, account status.	One profile; may submit feedback; receives notifications.
CommuterProfile	userId, full name, contact number, home/meeting area, workplace, preferred route, emergency contact, communication preferences, onboarding status.	Belongs to one User; may reference a preferred Route.
Route	id, origin, destination, route description or corridor.	Referenced by profiles and trips.
Operator	id, display name, vehicle/operator reference, contact details where appropriate.	Can be associated with many Trips.
Trip	id/reference, routeId, operatorId, scheduled time, vehicle details, status.	Belongs to a route; may have an operator; can have feedback records.
Feedback	id, tripId, commuterId, rating, category, comment, submittedAt, status.	Belongs to a Trip and submitting User; has response history.
FeedbackResponse	id, feedbackId, reviewerId, response, status change, createdAt.	Belongs to Feedback and an authorised reviewer.
Notification	id, userId, message/type, createdAt, readAt.	Belongs to a User.

Relationship summary: User 1—1 CommuterProfile; Route 1—many Trips; Operator 1—many Trips (optional where unknown); Trip 1—many Feedback records; User 1—many Feedback submissions; Feedback 1—many FeedbackResponses; User 1—many Notifications.

Implementation notes: use ASP.NET Core Identity or an equivalent established identity system; do not store plaintext passwords. Enforce foreign keys and useful indexes. The API must verify that a trip is completed and that the signed-in commuter is permitted to submit feedback for it. Use synthetic data in demonstrations.

3.3 Main API endpoint outline

Method & endpoint	Purpose	Access / rules
POST /api/auth/register	Create a user account.	Public; validate input and prevent duplicate accounts.
POST /api/auth/login	Authenticate a user and establish a secure session/token.	Public; rate-limit and return safe errors.
POST /api/auth/logout	End the authenticated session.	Authenticated; behaviour depends on cookie/token design.
GET /api/profile/me	Retrieve the current user's profile.	Authenticated; own profile only.
PUT /api/profile/me	Create/update profile and onboarding fields.	Authenticated; validate server-side.
GET /api/dashboard	Return onboarding status, trips, feedback, concerns, notifications.	Authenticated; data scoped to the user.
GET /api/trips	List trip records available to the commuter.	Authenticated; enforce data visibility rules.
GET /api/trips/{id}	Get trip details.	Authenticated; verify access to requested trip.
POST /api/trips/{id}/feedback	Submit a rating, category, and comment.	Authenticated; completed trip and eligibility required.
GET /api/feedback/mine	List the current user's feedback and statuses.	Authenticated; own submissions only.
GET /api/admin/feedback	Review submitted feedback.	Reviewer/Admin role required.
POST /api/admin/feedback/{id}/responses	Add an official response.	Reviewer/Admin role required; record response history.
PATCH /api/admin/feedback/{id}/status	Update review/resolution status.	Reviewer/Admin role required; validate allowed transitions.
GET /api/notifications	List notifications for current user.	Authenticated; own notifications only.
PATCH /api/notifications/{id}/read	Mark a notification as read.	Authenticated; own notification only.

3.4 Main user journey

1. Register & sign in User creates an account and authenticates securely.	**2. Complete profile** Add contact, route/workplace, emergency contact, communication preferences.
3. View dashboard Check onboarding status, scheduled/past trips, feedback, concerns and notifications.	**4. Submit feedback** After an eligible completed trip, provide a rating, category and comment.

5. Reviewer handles feedback Authorised reviewer responds, updates status and records resolution.	**6. User checks outcome** Dashboard/feedback history shows the response and current status.

Pay-iLift extension (subject to reviewer approval): drivers publish a commute schedule and available seats; commuters search compatible origin/destination, workplace and time; commuters request a lift; drivers accept or decline. This would require additional entities such as DriverProfile, CommuteSchedule, RideRequest and TripPassenger. It should not displace the required baseline POC workflows.

Design decisions to confirm with the reviewer: cookie-based session versus bearer-token authentication; exact role definitions; whether route/operator records are seeded demonstration data or admin-managed; allowed feedback status transitions; and whether the lift-matching extension is in scope.

Document prepared as a planning and design baseline. Confirm scope and deployment expectations with the reviewer before implementation.